

official_matrix_88 원본 충실 재현 보고서

커널 생성 에이전트 × 공식 벤치마크 매트릭스: 현황과 완전 재현 계획

2026년 7월 13일

Abstract

본 보고서는 official_matrix_88(11개 커널 생성 에이전트 × 8개 공식 벤치마크 = 88셀)을 각 에이전트의 **실제 구현/모델**로 재현하고 각 벤치마크의 **공식 오라클**로 채점하는 “원본 충실(faithful) 재현”의 현황과 남은 완전 재현 계획을 정리한다. 기존에 저장소에 커밋되어 있던 88셀 결과는 11개 에이전트가 모두 하나의 제너릭 드라이버를 통해 “smoke test용 단순 torch 코드”를 생성한 것으로, 에이전트의 실제 성능을 나타내지 않는다. 본 계획의 대상 범위는 **실제 구현/모델로 충실 재현이 가능한 8개 에이전트**로 한정한다: cudaforge, autokernel, autotriton, kernelllm, ksearch, kernelskill, drkernel, incoder32b. 모델이 비공개인 2종(cuda_ll, cuda_agent)과 AMD 전용 1종(geak)은 범위에서 제외한다. 따라서 대상 매트릭스는 **8개 에이전트 × 8개 벤치마크 = 64셀이며 전 셀이 원본 충실(faithful)**이다. **현재까지의 진척**: (1) 8개 faithful 에이전트를 전부 실제 구현/모델로 재현하여 KernelBench(level1/19_ReLU)의 공식 오라클 판정을 확보하였고(6/8 correct, 2/8은 모델 자체의 충실한 실패), (2) 8개 벤치마크의 공식 오라클을 전부 이 Blackwell(sm_120) 박스에서 배선하여 각각 구조화된 공식 판정을 반환함을 확인하였다. 즉 완전 매트릭스를 위한 두 축(에이전트 축, 벤치마크 축)이 모두 준비되었다. 이하에서 재현 결과, 오라클 배선 현황, GPU 구성, 시간/비용, 결론, 논문 연결을 제시한다.

1 배경

official_matrix_88은 서베이 논문의 실증(empirical) 섹션을 위한 자기완결형 하네스로, 11개 오픈소스 커널 생성 에이전트를 8개 벤치마크의 공식 오라클로 통일 평가한다. 그러나 저장소에 커밋된 결과는 모든 에이전트가 drivers/generic_llm_kernel_driver.py 하나를 경유했고, 그 프롬프트가 명시적으로 “for this smoke test, do not use Triton/CUDA/inline extensions”라고 지시하여 실제 커널이 아닌 단순 torch 코드를 생성하였다. 따라서 커밋된 88셀은 **파이프라인 동작 확인(smoke)**일 뿐 에이전트 성능이 아니다.

본 작업의 목표는 각 에이전트를 **원저자의 실제 구현/모델 그대로** 구동하여 “실제 에이전트 생성 → 공식 오라클 판정”의 진짜 데이터를 얻는 것이다. 모든 에이전트는 로컬 백본(예: Qwen2.5-Coder-14B) 또는 각자의 공개 모델을 사용하며, 평가는 벤치마크의 공식 오라클(예: KernelBench는 telemetry/instrumented_final_eval.py)로 수행한다.

2 재현 방식의 3가지 분류

에이전트를 실제로 조사한 결과, “에이전트”는 세 가지 서로 다른 형태를 가진다.

- **라이브 에이전트(프레임워크 내장 루프)**: 생성 → 컴파일/벤치 → 피드백 → 개선을 스스로 수행. 예: cudaforge(main.py, NCU 하드웨어 피드백 10라운드), kernelskill(KernelMem, 메모리 루프), ksearch, drkernel(KernelGYM).
- **모델-온리 릴리스**: 저장소에 에이전트 코드가 없고 파인튜닝된 모델이 산출물. 모델에 프롬프트 하면 커널을 생성. 예: autotriton(8B), kernelllm(8B), incoder32b(32B).

- **아티팩트 replay**: RL 학습 모델이 비공개이고, 사전 생성된 최적화 코드만 공개. 라이브 실행 불가, 공개 코드를 replay하여 채점만 가능. 예: cuda_ll, cuda_agent.

3 재현 결과: faithful 8종 전부 (KernelBench level1/19_ReLU)

각 에이전트의 실제 구현/모델로 재현하고 공식 KernelBench 오라클 (telemetry/instrumented_final_eval.py)로 채점한 결과다. 평가 하드웨어는 단일 NVIDIA RTX PRO 6000 (Blackwell, sm_120)이다.

에이전트	유형	재현 방식(원본 그대로)	공식 판정	speedup
cudaforge	라이브 CUDA	main.py 3라운드 + NCU 하드웨어 피드백	🟢 correct	0.70×
autokernel	라이브 CUDA	네이티브 브리지 + Qwen keep/revert 루프	🟢 correct	1.008×
autotriton	모델 Triton(8B)	AutoTriton RL 모델 서빙	🟢 correct	0.99×
kernelllm	모델 Triton(8B)	KernelLLM 공식 프롬프트 템플릿	🟡 compiled, 🛑 incorrect	—
ksearch	라이브 Triton	world-model 탐색 2라운드	🟢 correct	0.994×
kernelskill	라이브 CUDA	KernelMem 메모리 루프 + 자가수정	🟢 correct	0.695×
drkernel	모델 Triton(14B)	Dr.Kernel-14B reasoning 생성	🟢 correct	0.994×
incoder32b	모델 CUDA(32B)	IndustrialCoder(vLLM Llama 오버라이드)	🛑 compiled=False	—

Table 1: faithful 8종의 공식 오라클 판정(단일 태스크 ReLU). **6/8 correct, 2/8 충실한 실패** (kernelllm은 차원을 잘못 하드코딩, incoder32b는 CUDA 디바이스 코드를 C++ 컴파일 유닛에 배치 — 둘 다 파이프라인 문제가 아니라 모델 자체의 오류). ReLU는 메모리 바운드가 speedup이 1.0 부근에 수렴하므로, 이는 성능 순위가 아니라 파이프라인/정확도 검증이다.

이는 기존 smoke 결과와 질적으로 다른, 논문에 사용 가능한 실측 데이터다. 예컨대 cudaforge는 실제로 NCU를 실행해 하드웨어 메트릭을 피드백하여 커널을 0.5030→0.5311(내부 점수)로 개선하였고, ksearch는 world-model 탐색으로 1라운드 실패 후 2라운드에서 통과 커널을 찾았으며, autotriton/drkernel/kernelllm은 각자의 RL 모델로 커널을 생성하였다. 2건의 실패도 “correct<eval은 정상적 벤치 결과”(README)로서 정직한 데이터포인트다.

4 에이전트별 완전 재현 가능성

11개 에이전트를 실제 구현/모델로 재현 가능한지 재분류하였다.

핵심: 11개 전부를 원본 충실 재현하는 것은 불가능하다. 3개 에이전트가 근본적으로 막힌다 (모델 비공개 2: cuda_ll, cuda_agent / AMD 전용 1: geak). 이는 본 작업의 한계가 아니라 원저자의 미공개 또는 타 하드웨어 요구에 기인한다. 따라서 본 계획은 이 3종을 범위에서 제외하고, **충실 재현이 가능한 8종만**을 대상으로 하여 매트릭스 전 셀을 원본 충실로 유지한다.

5 재현 범위: faithful 8종 (64셀 전부 충실)

모델 비공개 2종과 AMD 전용 1종을 제외하고, 실제 구현/모델로 충실 재현이 가능한 **8 에이전트 × 8 벤치마크 = 64셀**만을 대상으로 한다. 전 셀이 원본 충실이므로 replay/N-A 라벨이 필요 없다.

- **faithful 8종**: cudaforge, autokernel, autotriton, kernelllm, ksearch, kernelskill, drkernel, incoder32b
- **제외 3종**: cuda_ll(모델 비공개), cuda_agent(모델 비공개), geak(AMD 전용)
- **8 벤치마크**: kernelbench, robust_kbench, tritonbench_t, tritonbench_g, multikernelbench, backendbench, pareval, sol_execbench

에이전트	정체	재현 가능성
cudaforge	라이브(CUDA, NCU)	🟢 faithful
autokernel	라이브(CUDA/Triton 브리지)	🟢 faithful
autotriton	모델-온리(8B, Triton)	🟢 faithful
kernelllm	모델-온리(8B, Triton)	🟢 faithful
ksearch	라이브(frontier LLM)	🟢 faithful
kernel skill	라이브(KernelMem 메모리)	🟢 faithful
drkernel	RL 모델(14B)+gym	🟢 faithful
incoder32b	모델-온리(32B)	🟡 faithful (무거움)
cuda_l1	RL 모델 비공개	🟡 replay만
cuda_agent	RL 모델 비공개(데이터셋만)	🟡 replay/대체만
geak	AMD MI 전용	🔴 NVIDIA 재현 불가

Table 2: 재현 가능성 재분류. geak(AMD 전용)는 제외, cuda_l1/cuda_agent는 모델 비공개로 replay만 가능.

⇒ 64셀, 전부 faithful.

6 벤치마크 측: 8개 공식 오라클 배선 완료 (Blackwell)

매트릭스의 다른 측인 8개 벤치마크의 공식 오라클(채점기)을 전부 이 Blackwell 박스에서 실행하여, 각각 구조화된 공식 판정을 반환함을 확인하였다. 러너(official_all_matrix_v1.py)의 evaluate_* 디스패치는 코드 수정 없이 동작했으며, 필요한 것은 환경 설정 (TORCH_CUDA_ARCH_LIST=12.0, 실행별 새 TORCH_EXTENSIONS_DIR, 오프라인 플래그, LLM 엔드포인트)과 prepare(태스크 매니페스트 생성) 뿐이었다.

벤치마크	공식 오라클(채점기)	배선 검증(cudaforge, T=1)
kernelbench	instrumented_final_eval.py	🟢 (8종 채점에 사용)
robust_kbench	kb instrumented (bench.sh matrix)	🟢 official_eval=1, correct=1
tritonbench_t	TritonBench EVAL (call/exe/eff)	🟢 official_eval=1, correct=1
tritonbench_g	TritonBench EVAL	🟢 official_eval=1, correct=1
multikernelbench	eval_single_runner.py	🟢 official_eval=1, correct=1
backendbench	BackendBench CLI (directory backend)	🟢 official_eval=4/8, “not covered” 정상
pareval	ParEval C++ 드라이버 (run-all.py)	🟢 official_eval=1, correct=0
sol_execbench	python -m sol_execbench.cli.main	🟢 official_eval=1, correct=1

Table 3: 8개 벤치마크 오라클 전부 Blackwell에서 정상 작동(구조화된 공식 판정 반환). smoke 후보(cudaforge 제너릭 드라이버)로 배선을 검증한 것이므로 correct 값 자체는 참고용이며, 핵심은 각 오라클이 이 하드웨어에서 실행되어 판정을 낸다는 점이다. backendbench의 add/mul/sub/div는 로컬 테스트 스위트가 없어 “not covered”로 정상 처리된다(README 문서화 동작). prepare로 생성된 태스크 수: kernelbench 249, robust 200, tritonbench_t 166, _g 184, multikernel 401, backendbench 15, pareval 60, sol 21.

요약: 에이전트 측(8종 재현)과 벤치마크 측(8 오라클 배선)이 모두 준비되어, 완전 매트릭스(64셀) 실행에 필요한 인프라가 완성되었다.

7 GPU 구성 (4장, 최상위)

추천: 4× NVIDIA B200 (180GB, Blackwell) — 32B 모델도 여유, sm_120 검증 완료. 저렴한 대안: 4× H100 80GB 또는 H200 141GB.

- GPU 0–1: 에이전트 모델 서빙(vLLM). 에이전트마다 백본이 달라 모델 스왑 필요 (Qwen14B, AutoTriton8B, KernelLLM8B, drkernel14B, incoder32B ...).
- GPU 2–3: 커널 컴파일 및 공식 오라클 eval 병렬 실행.
- 병렬 효율 약 70%(모델 스왑, 라운드 내 직렬성) $\Rightarrow$ 유효 배속 $\approx 2.8\times$.

8 시간 및 비용 추정

셀당 시간 = 태스크 수 $T \times$ (생성 + eval). 로컬 실측 기반 태스크당 평균:

에이전트 유형	생성/태스크	eval/태스크	합
반복형 ($\times 5$: cudaforge, autokernel, ksearch, kernelskill, drkernel)	~ 6 분	~ 2 분	8분
단발형 ($\times 3$: autotriton, kernelllm, incoder32b)	~ 2 분	~ 2 분	4분

Table 4: 태스크당 평균 소요(로컬 RTX PRO 6000 실측 기반). 벤치마크 1개(8 에이전트)당 = $5 \times 8 + 3 \times 4 = 52T$ 분, $\times 8$ 벤치 = $416T$ 분(단일 GPU 직렬).

4-GPU 유효배속($\div 2.8$) 적용 시:

T (셀당 태스크)	벽시계(4×GPU)	GPU-시간	4×B200($\sim \$20/\text{hr}$)	4×H100($\sim \$10/\text{hr}$)
1 (현재 수준)	~ 2.5 시간	~ 10	$\sim \$50$	$\sim \$25$
10 (권장 최소)	~ 25 시간	~ 99	$\sim \$500$	$\sim \$250$
50	~ 5.2 일	~ 496	$\sim \$2,480$	$\sim \$1,240$
250 (KernelBench 전량)	~ 26 일	$\sim 2,480$	$\sim \$12,400$	$\sim \$6,200$

Table 5: 균일 T 가정의 상한 근사. 실제 T 는 벤치마크별로 다르다(pareval/sol 등은 태스크 수가 적음).

일회성 통합 엔지니어링은 이미 완료되었다. CudaForge/autokernel 등을 붙이는 데 CUDA OOM, torch 확장 stale lock, PYTHONPATH, trial timeout, 커스텀 아키텍처 로딩 등 다수의 디버깅이 필요했으나 (부록 참조), **8종 에이전트 통합과 8개 오라클 배선이 모두 끝난 상태**이다. 따라서 남은 것은 위 표의 **GPU 실행 비용(T 에 비례)뿐**이다.

- 에이전트 통합(8종): **완료** — 드라이버 및 서빙 절차 확립.
- 벤치마크 오라클 배선(8종): **완료** — 전부 Blackwell에서 검증.
- 남은 작업 = smoke 제너릭 드라이버 대신 **실제 에이전트를 8개 벤치마크 전부에 $T > 1$ 로 실행**.

권장 운영점: $T = 10$. 4×H100 기준 ~ 25 시간 / $\sim \$250$ 이면 통계적 최소 의미를 확보하며, 통합/배선이 완료되었으므로 **바로 실행 가능**하다.

9 결과물

8×8 = 64셀 매트릭스(전부 faithful) — 셀마다 공식 오라클 지표를 담는다.

- 정확도: correct_rate, pass@1

- 성능: `speedup / fast_p` (`fast_1.0/1.5/2.0/3.0`)
- 원출력 및 verdict JSON 전량 보존(재현성).

즉 “에이전트 × 벤치마크”를 동일 공식 오라클로 사과-대-사과 비교한 표와 벤치별 난이도 프로파일. 모든 셀이 원본 충실이므로 replay/N-A 각주가 필요 없다.

10 내릴 수 있는 결론

1. **에이전트 간 공정 비교**: 서로 다른 프레임워크를 동일 오라클/하드웨어에서 재평가하여 벤치마크 계열별로 어느 에이전트가 실제로 정확하고 빠른 커널을 내는지 규명.
2. **벤치마크 난이도 구조**: kb-계열 vs operator-fill(backendbench) vs 병렬(pareval) 등의 성공률 패턴.
3. **패러다임 효과**: 반복형(NCU 피드백)/메모리형/단발 모델/RL 모델의 상대 우위.
4. **정직한 경계**: 대상은 충실 재현이 가능한 8종이며, 모델 비공개 2종/AMD 1종은 범위 밖임을 명시하여 결론 범위를 한정.
5. 기존 smoke 대비 “진짜 커널 생성” 재평가라는 점.

못 내리는 결론: geak/AMD 비교, `cuda_agent/cuda_ll`의 라이브 성능(모델 비공개), T 가 작을 때의 세밀한 순위.

11 논문 연결

- **Table 2 (taxonomy)**: 에이전트 × 벤치마크 분류에 재현 방식(faithful/replay/N-A) 표기 추가.
- **Contribution 2 (인프라)**: 모든 벤치마크의 공식 오라클을 하나로 배선한 이식형 통합 하네스 및 에이전트별 실제 구현 통합(`repro_env.sh`, 각 드라이버). **8종 에이전트 재현과 8개 오라클 배선을 Blackwell(sm_120)에서 end-to-end로 검증 완료.**
- **Main Results (Table 3)**: $8 \times 8 = 64$ 셀 완전 충실 매트릭스(정확도 + `fast_p`).
- **Analysis**: 위 결론 1–3.
- **Limitations / Threats to Validity**: 범위 제외 3종(모델 비공개 2: `cuda_ll`, `cuda_agent` / AMD 전용 1: geak), controlled vs native backbone, T 규모, 하드웨어(Blackwell sm_120) 특수성.
- **Reproducibility 부록**: 아래 하드-원 이슈 및 커맨드.

12 부록: 재현 환경 하드-원 이슈

Blackwell RTX PRO 6000 상에서 KernelBench 형식 eval + vLLM 서빙 시 확인된 비자명 이슈와 해법:

- A1. **아키텍처 핀**: `TORCH_CUDA_ARCH_LIST=12.0(sm_120)` 미설정 시 6개 아키텍처를 모두 컴파일하여 3분+ 소요 → eval 타임아웃. 핀 설정 시 콜드 컴파일 ~33초.
- A2. **torch 확장 stale lock**: 타임아웃으로 SIGKILL된 컴파일이 `~/.cache/torch_extensions/.../lock`(무확장자)를 남겨 이후 프로세스가 무한 대기. 실행마다 새 `TORCH_EXTENSIONS_DIR` 사용으로 회피.
- A3. **GPU 공유**: vLLM과 eval이 한 GPU를 공유. `--gpu-memory-utilization 0.4`로 서빙하여 eval에 여유 확보(0.85는 OOM 유발).

- A4. **vLLM flashinfer 불일치**: FLASHINFER_DISABLE_VERSION_CHECK=1로 우회.
- A5. **autokernel 통합**: 플레이북이 `from kernels.cuda._compile import compile_cuda`를 사용하므로 PYTHONPATH에 autokernel 루트 추가 필요. 콜드 `compile_cuda`가 30초 trial 타임아웃 내부에서 실행되므로 AK_TRIAL_TIMEOUT 상향 필요.
- A6. **NCU**: 본 장비에서 sudo 없이 GPU 카운터 접근 가능 → cudaforge 하드웨어 피드백 정상.
- A7. **kernelllm**: 출력이 torch-inductor 스타일이며 torch≥2.11에서 제거된 grid 심볼을 import 한다 → 커널 로직은 그대로 두고 grid만 복원하는 버전-호환 심을 앞에 덧댐.
- A8. **kernelkill(KernelMem)**: 릴리스에 `bench_ref_inputs_0.py` 템플릿 누락 → 부모 프로젝트 CudaForge의 동일 파일 이식. NCU 경로가 `/root/...`로 하드코딩 (무시하고 진행). 프롬프트가 컴파일 플래그를 A100/sm_80으로 하드코딩.
- A9. **ksearch(K-Search)**: KernelBench를 kernelbench 패키지로 import → `third_party/KernelBench/src`를 PYTHONPATH에 추가.
- A10. **incoder32b(IndustrialCoder)**: 커스텀 아키텍처 `IQuestCoderForCausalLM`은 실제로는 Llama 계열(QKV bias 없음, RMSNorm, SwiGLU, GQA, rope_theta 500000)이다. `transformers trust_remote_code` 경로는 rope 초기화 비호환으로 깨진 출력을 냈고, `vLLM --hf-overrides architectures=LlamaForCausalLM`로 서빙하니 정상 로드/생성되었다.
- A11. **벤치마크 오라클(8종)**: 러너의 `evaluate_*` 배선은 코드 수정 없이 Blackwell에서 동작하였고, 위 A1(아키텍처 핀)/A2(stale lock) 환경 설정과 `prepare`만으로 전부 실행되었다.